

LLD – Author Module (OpenScholar)

1. Overview

This document defines the Low-Level Design (LLD) for the Author module of OpenScholar. The module is responsible for managing authorship across paper versions, supporting both registered users and external contributors, ensuring correct ordering, and preventing data duplication.

The design ensures normalization, reusability of author entities, and consistency across versions.

2. Scope

The Author module is responsible for:

- Managing authors linked to paper versions
- Supporting both registered users and external authors
- Maintaining author order per paper version
- Reusing existing author records when applicable
- Preventing duplicate author entries

Out of Scope:

- Paper creation logic (handled in Paper module)
 - User authentication (handled in Auth module)
-

3. Data Model (Relevant Tables)

Tables used:

- authors
- paper_authors
- users

authors

- id (UUID)
- name (TEXT)
- user_id (UUID, nullable)

paper_authors

- paper_version_id (UUID)
- author_id (UUID)

- author_order (INT)

Constraints:

- PRIMARY KEY (paper_version_id, author_id)
 - author_order must be unique per paper_version
-

4. Core Design Principles

- A registered user can act as an author (linked via user_id)
 - External authors have no user_id
 - Authors should be reused if same identity exists
 - Author order is strictly maintained per version
-

5. Author Resolution Strategy

When creating or updating authors:

Case 1: Registered User Author

- If user_id is provided:
- Check if author exists with same user_id
- If exists → reuse
- Else → create new author record

Case 2: External Author

- If user_id is null:
 - Normalize name (trim, lowercase)
 - Check if author exists with same normalized name
 - If exists → reuse
 - Else → create new author record
-

6. API Contracts

Note: Author data is always passed within Paper APIs.

Author Input Format

```
{ "name": "Author Name", "userId": "uuid | null", "order": 1 }
```

Validation Rules:

- name must not be empty
 - order must be positive integer
 - order must be unique per request
-

7. Service Logic

7.1 resolveAuthors(authorsInput)

Steps: 1. Initialize empty list of author_ids 2. For each author input: a. If userId exists: - Find author by user_id - If not found, create new author b. If userId is null: - Normalize name - Find author by normalized name - If not found, create new author 3. Return list of author_ids with order

7.2 assignAuthorsToVersion(paperVersionId, authors)

Steps: 1. Validate author order uniqueness 2. Delete existing paper_authors for version 3. Insert new paper_authors entries

8. Validation Rules

- author_order must start from 1
 - author_order must be continuous (1,2,3,...)
 - no duplicate author_order
 - no duplicate author_id in same version
-

9. Security Considerations

- Only paper owner can modify authors
 - Validate user identity via JWT
-

10. Edge Cases

- Duplicate authors in input
 - Same user added multiple times
 - External author name variations (e.g., "John Doe" vs "john doe")
 - Missing or invalid order sequence
-

11. Prisma Model Mapping

```
model Author { id String @id @default(uuid()) name String userId String?
```

```
@@index([userId]) }
```

```
model PaperAuthor { paperVersionId String authorId String authorOrder Int
```

```
@@id([paperVersionId, authorId]) }
```

12. Folder Structure

```
src/ modules/author/service.ts modules/author/repository.ts
```

13. Implementation Notes

- Normalize author names before comparison
 - Use transactions when assigning authors to ensure consistency
 - Avoid duplicate author creation
 - Use batch queries for performance
-

14. Acceptance Criteria

- System correctly assigns authors to paper versions
 - Author order is preserved and validated
 - Registered users are reused as authors
 - External authors are deduplicated
 - No duplicate author entries created unnecessarily
-

15. Future Enhancements

- Author profile pages
 - ORCID integration
 - Author affiliation history
-

End of Document